前端模块化开发

简介

随着网站逐渐变成”互联网应用程序”，嵌入网页的 `Javascript` 代码越来越庞大，越来越复杂。

`Javascript` 模块化编程，已经成为一个迫切的需求。理想情况下，开发者只需要实现核心的业务逻辑，其他都可以加载别人已经写好的模块。但是，`Javascript` 不是一种模块化编程语言，它不支持”类”（class），包（package）等概念，也不支持”模块”（module）。

CommonJS规范

1、创建“module”文件夹

2、创建 mokuai-common-js/四则运算.js

```
// 定义成员：
const sum = function(a,b){
    return a + b
}
const subtract = function(a,b){
    return a - b
}
const multiply = function(a,b){
    return a * b
}
const divide = function(a,b){
    return a / b
}
```

3、导出模块中的成员

```
// 导出成员：
module.exports = {
    sum: sum,
    subtract: subtract,
    multiply: multiply,
    divide: divide
}
```

简写

```
//简写
module.exports = {
    sum,
    subtract,
    multiply,
    divide
}
```

4、创建 mokuai-common-js/引入模块.js

```
//引入模块，注意：当前路径必须写 ./
const m = require('./四则运算.js')
console.log(m)
const r1 = m.sum(1,2)
const r2 = m.subtract(1,2)
console.log(r1,r2)
```

5、运行程序

```
node 引入模块.js
```

CommonJS使用 exports 和require 来导出、导入模块。

ES6模块化规范

1、创建 mokuai-es6 文件夹

2、创建 src/userApi.js 文件，导出模块

```
export function getList() {
    console.log('获取数据列表')
}
export function save() {
    console.log('保存数据')
}
```

3、创建 src/userComponent.js文件，导入模块

```
//只取需要的方法即可，多个方法用逗号分隔
import { getList, save } from './userApi.js'
getList()
save()
```

> **注意：这时的程序无法运行的，因为ES6的模块化无法在Node.js中执行，需要用Babel编辑成ES5后再执行。**

4、初始化项目

```
npm init -y
```

5、配置 .babelrc

```
{
    "presets": ["es2015"],
    "plugins": []
}
```

6、安装转码器，在项目中安装

```
npm install --save-dev babel-preset-es2015
```

7、定义运行脚本，package.json中增加”build”

```
{
    // ...
    "scripts": {
       "build": "babel src -d dist"
    }
}
```

8、执行命令转码

```
npm run build
```

9、运行程序

```
node dist/userComponent.js
```

ES6模块化写法2

1、创建 src/userApi2.js ，导出模块

```
export default {
    getList() {
        console.log('获取数据列表2')
    },
    save() {
        console.log('保存数据2')
    }
}
```

2、创建 src/userComponent2.js，导入模块

```
import user from "./userApi2.js"
user.getList()
user.save()
```

3、执行命令转码

```
npm run build
```

4、运行程序

```
node dist/userComponent2.js
```